

[Resubmission] Building Probabilistic Performance Guarantees for Composable Computing Services

No Author Given

No Institute Given

Abstract. The increasing complexity of optimizing service offers creates an exponential configuration space that is often too vast for exhaustive empirical benchmarking. Consequently, service providers struggle to offer diverse configurations without risking significant penalties for violated Service Level Objectives (SLOs). To address this, we propose a framework for providing Probabilistic Performance Guarantees (PPGs) for arbitrary service configurations and their compositions. Our methodology utilizes Gaussian Processes (GPs) to model the stochastic performance of individual services, enabling high sample efficiency and the quantification of epistemic uncertainty. For composite applications, we introduce a compositional framework based on Decoupled Variational Inference, which supports the propagation of performance guarantees across multi-stage service chains while preserving the autonomy of individual providers. We evaluate our approach using a physical prototype featuring three services for QR detection, object detection (YOLOv10), and Lidar processing. Our results demonstrate that the proposed framework accurately predicts performance across unseen configurations and introduces a “readiness score” to help providers balance expected benefits against the probabilistic risk of SLO violations. We release our implementation and results on <https://anonymous.4open.science/r/cao/README.md>.

Keywords: Service Level Objectives · Performance Guarantees · Gaussian Processes · Service Composition · Distributed Computing

1 Introduction

A single computing service can be offered, recomposed, and operated in a lot of different ways. For instance, an object detection service could operate with low resource overhead on an Edge device—trading off accuracy—or run in a Cloud server, where larger detection models permit maximum accuracy. A service provider can thus cover large and diverse sets of clients and their requirements—quantified through their Service Level Objectives (SLOs). As the use case evolves, e.g., request load increases, a service can move across this solution space; for autoscaling it has been shown [17] that a large and diverse space (i.e., multiple elasticity strategies) improves SLO fulfillment in dynamic environments.

The variety of solutions, however, creates a structural tension for the provider: the number of possible service offers (i.e., defined through parameters) is exponential with the number of parameter assignments, e.g., the object detection service could adjust the input tensor size (quality) and allocated CPU resources (cost) on a pixel, or nanosecond granularity. This vast solution space makes it impossible to empirically benchmark all possible service configurations, also because physically deploying a solution is costly. This prevents service providers from offering large parts of the solutions space because the uncertainty around these solutions could result in high penalties for violated SLOs. Consequentially, service providers are forced to cap the dimensionality of the solution space by constraining the number of customizable features [15,22], or heavily discretize the solution space [14,16]. These two workarounds greatly reduce the diversity—and thus robustness—of service offerings. However, to facilitate service composition at machine speed [1,21], it is not enough to serve a small set of configurations, targeted to a few user profiles; instead, we are looking at an exploding number of agents that will assemble applications at real-time. Hence, service providers require new mechanisms to support this process.

To increase the diversity of service offers and their compositions, service providers need to be able to give probabilistic performance guarantees (PPGs) for arbitrary service configurations. This paper addresses this gap through a methodology that allows interpreting the behavior of computing services according to its current configuration, and in further consequence, make predictions about the performance under arbitrary service configurations or compositions. Our model is based on a Gaussian process, which reflects the uncertainty of real-world environments, allows extrapolating from existing knowledge, and supports gradual updates as more data becomes available. To predict the performance of composite service chains, our methodology allows concatenating the individual service models to analyze how the performance guarantees propagate across the application. We evaluated our approach by analyzing three physical services across different SLO profiles and show how our approach can quickly identify a set of service offers that can be provided with high probabilistic guarantees. Hence, the contributions of this article are:

1. A probabilistic modeling approach for providing performance guarantees around arbitrary service configurations. Given a set of client preferences, e.g., client x prefers low cost, our approach infers pareto-optimal configurations; thus, supporting service providers and consumers in making agreements.
2. A compositional framework that allows to give performance estimates around multi-stage service chains. Our approach—based on Decoupled Variational Inference—preserves the autonomy of individual services as service stages are optimized independently, without any centralized orchestrator.
3. A physical prototype and evaluation that created probabilistic performance estimated around three different computing services, as well as their compositions. We published our prototype and results, where we discovered the *readiness score*: a suitable metrics for comparing the expected benefit from offering a service, against the potential cost of violating the service.

2 Background & Related Work

The proliferation of microservices and the Computing Continuum has shifted the focus from static resource allocation to dynamic service orchestration. This section examines the current state of evaluating service performance and the limitations of existing performance guarantee models.

2.1 Evaluating Service Performance

In service-oriented architectures, performance is typically governed by Service Level Objectives (SLOs). Generally, an SLO q relates a performance metric to a target value t_q ; for example, maintaining service throughput (tp) ≥ 30 . Given the increasing resource constraints of Edge environments [11], it is often infeasible to ensure optimal *throughput*; hence, traditional binary fulfillment models (satisfied/violated) fail to capture the nuances of finding tradeoffs for Edge computing services. Hence, we follow recent trends in flexible SLO management [10,17] and calculate the SLO fulfillment (ϕ^1) as a continuous value through Eq. (1).

$$\phi(q, m) = \begin{cases} \frac{m}{t_q} & \text{if } m < t_q \\ 1.0 & \text{if } m \geq t_q \end{cases} \quad (1)$$

While continuous fulfillment provides better granularity for optimization, it assumes a deterministic relationship between configuration and performance. In reality, shared infrastructure and fluctuating workloads introduce significant stochastic noise. Existing autoscaling approaches, such as MagicScaler [14] or reinforcement learning agents [18], attempt to learn these dynamics through iterative environment interaction. However, such methods require exhaustive “warm-up” periods and are often hindered by the limitations of static profiling [22], which fails to evolve during runtime. This sample-inefficient nature is impractical for modern containerized services with vast configuration spaces, where the time required for empirical benchmarking far exceeds the window for real-time service recomposition. Furthermore, these black-box models typically lack a quantifiable confidence measure, preventing providers from assessing the risk of SLO violations before a configuration is actually deployed.

2.2 Probabilistic Performance Guarantees

Providing performance guarantees in distributed systems traditionally relies on Queuing Theory [8] or Worst-Case Execution Time (WCET) analysis [20]. While these offer hard guarantees, they are often overly pessimistic for soft-real-time applications like video analytics or IoT stream processing. Consequently, researchers have turned toward Probabilistic SLA (P-SLA) modeling.

Recent work has explored using Bayesian frameworks and Markov Blankets to model service dependencies and uncertainty [16]. However, these models often

¹ We choose the letter ‘ ϕ ’ due to its sound: SLO ful-*phi*-llment

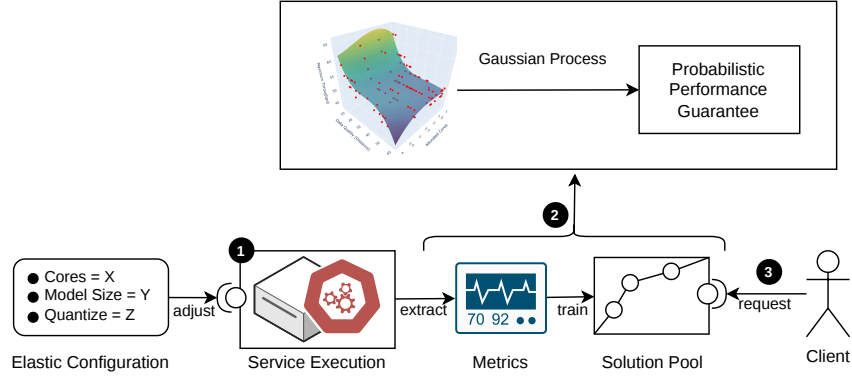


Fig. 1: High-level methodology of our framework: ① services are monitored under different configurations, ② we use the metrics to model the expected service behavior, and ③ we create probabilistic performance guarantees for different service offers.

discretize the state space, limiting their ability to generalize to unseen configurations. Gaussian Processes (GPs) have emerged as a powerful tool for regression in such domains due to their ability to provide not just a point estimate, but a full probability distribution (mean and variance) for any given input.

In the context of service composition, the challenge intensifies. When services are chained (e.g., a QR detector feeding into a data aggregator), the performance uncertainty of the first stage propagates to the next. Deep Gaussian Processes (DGPs) [2] offer a mathematical foundation for this propagation. However, standard DGP training requires global visibility, which violates the autonomy of independent service providers. Our work builds upon the concept of Decoupled Variational Inference to allow providers to maintain “local” performance models that can be mathematically composed into “global” chain guarantees without exposing internal service telemetry.

3 Methodology

When creating PPGs, our requirements are the following: the method must reflect the uncertainty in the processing environment, it must be highly sample-efficient, and it must allow extrapolating from the few empirical samples available. To that extent, we decide to create performance guarantees by using Gaussian Processes (GPs), which can model both individual services, as well as their compositions. Figure 1 shows the high-level overview of our methodology, which consists of ① monitoring the service execution under different configurations, ② training GPs from the service metrics, and ③ using the GPs and their compositions to infer a set of pareto-optimal candidates that can be offered with high performance guarantees. In the following, we explain its parts in more detail.

3.1 Service Configuration and Monitoring

An individual computing service s can be offered in a myriad of different ways: for example, a common DNN inference service can be offered with different model sizes or data qualities, while the maximum amount of resources is commonly determined by the maximum cost a consumer is willing to pay [18]. To support fine grained management of allocated resources, we assume that the service s is wrapped in an individual container (e.g., Docker), which exposes an API that allows changing the service configuration during runtime [17], e.g., to scale the provisioned resources, or any quality-related parameter. Finally, every second, the services’ performance metrics are scraped by a time-series DB (e.g., Prometheus)—we will use these metrics for training the GPs.

3.2 Gaussian Processes Modeling and Composing

A Gaussian process (GP) provides probabilistic estimates of the underlying distributions of its training data—the observed service metrics. As such, it can be used for modeling the behavior of an individual service—given that we adjust its configuration parameters in a certain way—as well as for composite service chains, where one service serves as input to its successors. Given its training data, the GP can express high certainty between highly-explored areas, and low certainty within unknown areas. Notice, that by design, GPs are not limited in the number of features (i.e., configuration parameters) they provide; however, their performance is said to decline for > 20 variables [2].

Model Fitting Our core assumption is that with increasing *resources* and lower *quality*, the service *throughput* will be higher; hence, the core aspect of our GPs is to predict the expected application *throughput* given the amount of *resources* provisioned and the configured *quality*, as expressed in Eq. (2). For simplicity, we assume the absence of any other dependencies between these three competing aspects; the absence or presence of these structural dependencies could also be analyzed through conventional structure learning algorithms [16].

$$K = \{\text{resources}, \text{quality}\} \rightarrow \text{throughput}. \quad (2)$$

By design, GPs are non-parametric, so they always depend on the training data. Fitting the process to the data has a complexity of $\mathcal{O}(n^3)$; for making predictions—like, inferring the expected *throughput*—getting a mean for a point is $\mathcal{O}(n)$, and getting its *sd* is $\mathcal{O}(n^2)$. To decrease this complexity, common GP libraries (e.g., pytorch) use *inducing points*—a subset of the most-representative samples that are kept within the GP, while other samples are ignored. Given the increasing numbers of metrics produced by the services, this will allow us to retain the most important samples, while decreasing the complexity.

Model Composition An individual service GP allows estimating the performance (and in further consequence SLO fulfillment) across different service configurations, regardless of whether they have been empirically explored already. More commonly, services are composed into larger applications, comprising hundreds of microservices, like the social network application in [4]. The question is thus, *how does the predicted performance of one service impact downstream services?*, which can be answered through Deep Gaussian Processes (DPGs)—a model, where one GP provides a normal distribution as an input feature to the GP in the next layer. Hence, additionally to any existing configuration parameters (i.e., *resources* and *quality*), every GPs that depends on upstream services has additional features for incoming distributions.

To provide accurate predictions across a service chain, standard DGPs typically optimize these multiple layers through a global backpropagation pass. This, however, requires a single entity to maintain the state of every stage; to improve the autonomy of individual service providers, our approach utilizes Decoupled Variational Inference: by defining computational boundaries and training in localized chunks via the Variational ELBO, we can optimize service stages independently or in small clusters. This allows composing these models into larger, more complex service structures without the overhead or architectural constraints of a centralized global state. The complexity of recomposing services from existing GPs is $\mathcal{O}(S)$, with S being the number of services in the chain. This reflects the fact that we do not perform any joint training across the GPs; hence, the computational complexity of recomposing the services’ GPs is independent from the actual size of their training data, except for storing (i.e., replicating) it. In the next step, we will show how to create a PPG from these models.

3.3 Probabilistic Performance Guarantees

Finally, to create PPGs, we use the knowledge gained from modeling the GPs. Consider a potential service consumer that considers whether a service can fulfill its requirements—specified through a set of SLOs Q . The question is hence, *given these SLOs, how likely is it that the service will be able to fulfill them?* However, consider that a service cannot fulfill competing requirements, e.g., low *resources* and high *throughput*; hence, solving this problem requires building a pareto front of solutions that characterize the performance tradeoffs.

We acknowledge that there exist many possible options for inferring pareto-optimal candidates, e.g., genetic algorithms or beam search, and refrain from making hypothesis about their superiority over each other. By preference, in our prototype, we will stick to a numerical solver that uses gradient descent to navigate the GPs and converge towards optimal service configurations.

3.4 Limitations

We make the following assumptions to mark the boundaries of our work:

- Our method assume that there are polynomial relations between the *resources*, *quality*, and *throughput*, where the consequential *throughput* can be represented through a Gaussian normal distribution. We did not test our methodology for other kinds of variable relations and refrain from making predictions how suitable this methodology would be in that case.
- Technically, a DGP can represent arbitrary tree-like hierarchies, and thus, model complex microservice compositions. However, owed to limited support in existing frameworks (e.g., gpytorch [5]) and overall technical complexity, our current methodology and evaluation targets only linear service chains.

4 Evaluation

The evaluation of our methodology analyzes all its logical steps: monitor the performance of processing services, model them as Gaussian processes, and use the individual services, or their compositions, for inferring performance guarantees. In particular, we implement a prototype of our framework and a custom evaluation environment; afterward, we analyze the accuracy and fitness of the performance guarantees according to different experience levels.

4.1 Evaluation environment

In the following, we describe how we implemented different parts of the methodology and which services we used to create the performance metrics. For executing the experiments, we also define a set of SLOs that mark highly orthogonal preferences, and the physical device we used for our evaluations.

Framework Implementation We train the GPs using pytorch (@2.11.0) and gpytorch (@1.15.2). We share our code repository and all our experimental results on <https://anonymous.4open.science/r/cao/README.md>.

Service Implementations Our methodology requires computing services that can be offered in many different ways (i.e., defined by their configuration parameters). To that extent, we use the following three processing services, which can all be monitored and reconfigured during runtime. Fig. 2 shows the output for each service, namely, we implement: (a) a QR code reader that detects QR code within video frames through OpenCV [13]; (b) a CV service that uses Yolov10 [19] for classifying objects in video frames; and (c) a point cloud renderer based on the Kitti dataset [6] that draws a mobile map around a moving vehicle.

Table 1 provides further details about the processing services, in particular, about their configuration parameters: the *QR* and *PC* services can both be served with more CPU *resources* (incurring higher costs) and variable *quality* rates; for the *QR* service, the *quality* refers to the respective video resolution, whereas the *PC* service can adjust the range of the mobile map. The *CV* service,

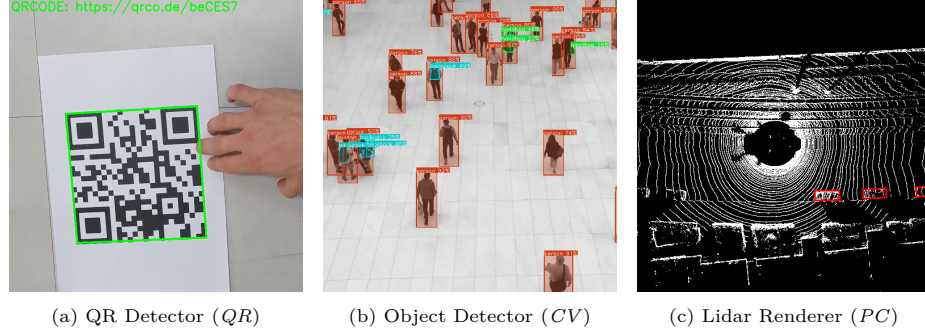


Fig. 2: Output of the three implemented processing services.

Table 1: List of processing services, that can each be adjusted in multiple configuration dimensions (*resources*, *throughput*, *quality*). We train Gaussian processes for each individual services and compose them arbitrarily into larger service chains.

ID	Service Description	Configuration Parameters
<i>QR</i>	Detect QR Code w/ OpenCV [13]	<i>resources</i> , <i>throughput</i> , <i>quality</i> (pixel)
<i>CV</i>	Object Detection w/ Yolov10 [19]	<i>resources</i> , <i>throughput</i> , <i>quality</i> (pixel & model)
<i>PC</i>	Lidar Point Cloud Processing [3]	<i>resources</i> , <i>throughput</i> , <i>quality</i> (range)

however, exposes two quality parameters, that allow adjusting the resolution of the input tensor, and the actual DNN model size used.

Service Level Objectives Instead of following a classical approach [12], where all service variables are related to a fixed threshold, we take the following approach: we summarize that consumers always want the best performance, i.e., the highest throughput, best quality, and all of that at the lowest cost. Therefore, instead of letting the consumer specify the actual SLO thresholds, we (1) extract the threshold from the actual performance metrics—thus, the service should always operate at its maximum practical *throughput*; and (2) we let the consumer only specify SLO weights, so how important each of the performance aspects is to them. From Eq. (2) we know that these aspects can be highly orthogonal, so it is impossible to fulfill all of them at the same time.

We summarize four sets of SLOs, which we present in Table 2. For instance, under **HIGH_PERF**, the service aims to optimize all three performance dimensions, but it gives 90% of weight to achieving the highest *throughput*. For a set of SLOs Q , where each SLO q features a weight w_q , the weighted SLO fulfillment for one service is calculated through

$$\sum_{j=1}^Q \phi_j \times w_j, \text{ with } \sum_{j=1}^Q w_j = 1. \quad (3)$$

Table 2: SLO Weights for relative service performance; services always aim to optimize each metric, the actual fulfillment is then graded according to the weight.

Perf. Metric	Different Sets of SLO Weights			
	DEFAULT	HIGH_PERF	LOW_COST	HIGH_QUALITY
↑ Throughput	0.34	0.90	0.05	0.30
↓ Resources	0.33	0.05	0.90	0.05
↑ Quality	0.33	0.05	0.05	0.65

Table 3: Physical device for executing the processing services and the methodology.

Full Device Name	Price ²	CPU	RAM	GPU
Lenovo ThinkStation PGX	5,800 €	ARM Grace 20C	128 GB	Blackwell

Hence, the overall weight of the three SLOs must sum up to 1. In cases where the performance aspect consists of multiple factors (e.g., the *quality* of the *CV* service), we equally split the weight between the factors; hence, *CV*’s *quality* is calculated from both the targeted model size and input resolution.

Notice, that the service *throughput* has a clear lower bound, i.e., producing no output at all. If any service configuration has an output of < 1 item / second, the overall SLO fulfillment is heavily penalized regardless of the *quality* or *cost*.

Physical Device To execute and monitor the processing services we use a physical device, a Lenovo PGX station; Table 3 contains the hardware specifications. We use the device for two purposes: (1) to execute the processing services and monitor their behavior across different configurations, and (2) to train the GPs based on these metrics; this involves GPU-heavy computations (e.g., fitting the GP to the data, or finding inducing points), making the PGX highly suitable.

4.2 Experimental Results

To analyze the maturity of our framework, we evaluate it through three experiments that target: (1) the accuracy and fitness of service offers inferred from an individual GP, (2.1) the uncertainty across increasingly larger service chains, and (2.2) the overhead of our approach for training GPs and running inference.

Experiment 1 – Individual Service Modeling Service providers and consumers must both know about the probabilistic guarantees surrounding a service offer; based on that, the parties can agree on a set of SLOs, a probability that

² Prices adopted from sparkfun and Lenovo official, accessed May 2026

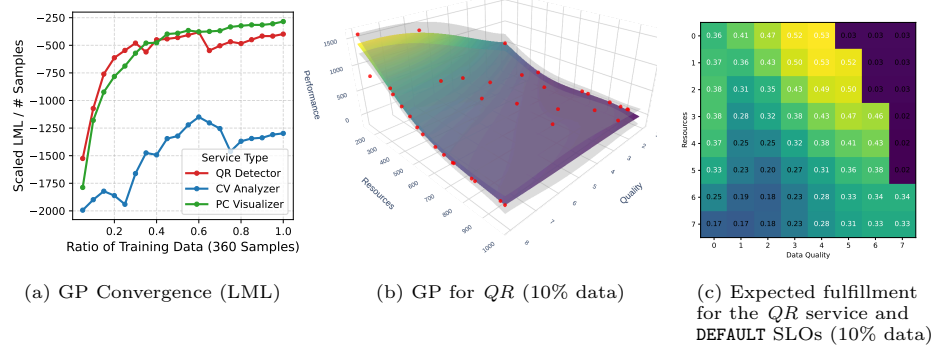


Fig. 3: Progress of the GP training and inference: (a) the LML rises continuously as more samples are used for training the GP, meaning the data explains the trend better; we also show (b) the shape of the GP for predicting the throughput of the *QR* service after seeing 10% of the data, and (c) the expected fulfillment for the **DEFAULT** SLOs.

they will be fulfilled, and a compensation for SLO violations. Thus, this experiment analyzes: *when does a provider have sufficient confidence to release a service?* and *what is a useful metric to decide if a service can be released?*

To answer these questions, we execute the three processing services on the PGX device, where they run completely isolated from each other. We monitor their performance every 10 seconds, and adjust the service execution randomly (i.e., uniformly sampling from services’ configuration spaces). Each service collects data for 6 minutes, which we repeat 10 times; this gives us 360 samples per service (= 6 min * 6 samples/min * 10 repetitions). This is the maximum amount of data used for training the GPs. We split the training data into 20 parts, and use increasingly more data (i.e., every iteration one split more) for GP training. In Fig. 3a we visualize the Log Marginal Likelihood (LML) of the training process as a metric for how well the GP can match the data. We observe that the LML³ constantly increases, meaning the GP generally fits the data well; only the *CV* service’s LML is lower, showing higher noise. To provide an intuition of how the trained GP looks like, Fig. 3b shows the GP for the expected *throughput* of the *QR* service after seeing 10% of training data. Notice, how the uncertainty is higher around areas that the agent has not yet explored well.

Next, we use the 20 GPs (i.e., with increasingly more training data) to infer the best service configuration at that time for each of the four SLO sets; we call these optimal configurations *candidates*, because they are highly suited to be permanently offered by the service provider. To find the optimal candidate for a GP $\times$ SLO combination, we use gradient descent within the GP and the SLOs as objectives. We start the numerical solver 25 times using the SLSQP [9] algorithm—every time using a new random starting position in the configuration space. Given that each sampling step in the GP gives a distribution for the

³ By default, the LML scales with the number of samples; to neutralize this factor and isolate the performance of GP training, we divide LML by the number of samples.

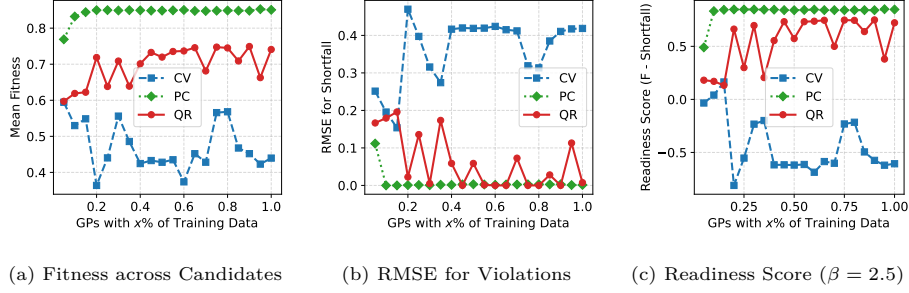


Fig. 4: Fitness of inferred candidate solutions across all sets of SLOs, combined with the RMSE for cases where SLOs were violated; the readiness score shown the tradeoff for expected benefit and penalty for offering a configuration.

expected performance, we use the 5th percentile to penalize areas with high uncertainty. For the DEFAULT SLOs, Fig. 3c show a heatmap of the expected SLO fulfillment across the configuration space; the evaluated candidate will likely come from the highest-scoring area.

Next, we actually deploy the inferred service configurations and repeat each configuration 50 times; this gives a total of 4000 samples per service (20 GP rates * 4 SLO sets * 50 repetitions). Each configuration runs for 10 seconds, meaning the entire experiment lasts for 40,000 seconds, or roughly 11 hours. For each GPs (i.e., with increasing training data) we compare the actual performance against the expected performance (as in Fig. 3c). We capture the actual fitness of the inferred candidate (Fig. 4a), the RMSE of prediction vs. reality when the performance was worse than promised (Fig. 4b), and combine these two factors into a *readiness* score (Fig. 4c), which is computed as

$$readiness := fitness - (rmse \times \beta),$$

thus comparing the expected benefit for releasing the services, against the cost of potential violations. Depending on the cost of violations, the penalization factor (β) can be adjusted; in our evaluation, we use $\beta = 2.5$ and assume that a configuration should be served if *readiness* is ≥ 0.5 . Across the GPs, we observe that the fitness of the selected candidates increases continuously, while the RMSE for the violations converged towards the lower end. Again, the *CV* service is the exception, showing that the GP struggles to ensure high certainty around the provided candidates. However, recall that the candidate configurations are not necessarily part of the general training data, e.g., when the service is offered in an unknown configuration. After manual inspection, we could confirm that this was the case; we deduce that our method requires an additional fine-tuning step as candidates are served continuously to update the expectation quickly.

To investigate the readiness across the different SLOs in more detail, we separate the global readiness score for the different SLOs, as shown in Fig. 5. We observe that even for the *CV* service there were some SLO sets (HIGH_PERF and LOW_COST) that can be readily served, whereas the large prediction errors mostly

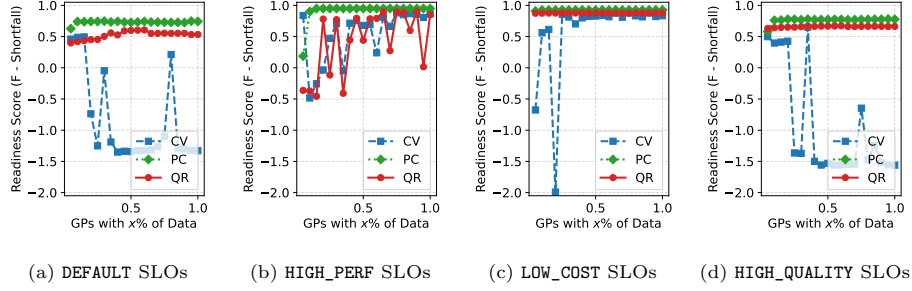


Fig. 5: Detailed readiness score for the individual sets of SLOs.

come from specific SLO sets (DEFAULT and HIGH_QUALITY). We conclude that the readiness score provides a useful metric for deciding if a service configuration can be served with high certainty, thus supporting service providers in estimating the maturity and guarantees around a service.

Experiment 2.1 – Service Composition To support dynamic service compositions during runtime, service providers and consumers must be able to predict the performance of arbitrary service chains, as individual links in the chain get replaced or recomposed. This experiment thus addresses the question of *what is the expected performance and certainty for an arbitrary service chain?*

In a service chain, the performance of individual links is usually determined by their previous link(s); to create such a scenario, we chain the three processing services in arbitrary permutations and inject the following bottleneck: a service cannot surpass the *throughput* of its previous service. We reuse the training data of the first experiment, where the services explored randomly, and perform the following modifications: (1) given that the services have vastly different *throughput*, we normalize the service *throughput* according to the empirical min-max measured in the dataset, and (2) inject the bottleneck directly to the data so that a service can never surpass its predecessor. From the resulting dataset, we use 80% for training the three GPs, while retaining 20% for testing purposes. We compose the services in three different combinations, as shown in Fig. 6. Notice, that if we repeat a service, we further split the test set across them to avoid identical performance. Given the composite models—the DGPs—we sample the expected *throughput* across the pipeline (=100 Monte Carlo samples) according to the configuration of each individual service.

For the three different chains, Fig. 6 shows the mean expected performance and standard deviation of each link. We compare the expected performance against the actual performance (test split) and plot the RMSE. We observe that in Fig. 6a the CV service clearly introduced a bottleneck for the remaining chain; the same can be observed in Fig 6b, but repeated CV services just increase the uncertainty around the remaining chain. This can be well perceived from the increasing RMSE across the chain. For Fig. 6c, where we only concatenated the same service, we can perceive a similar trend in terms of increasing uncertainty

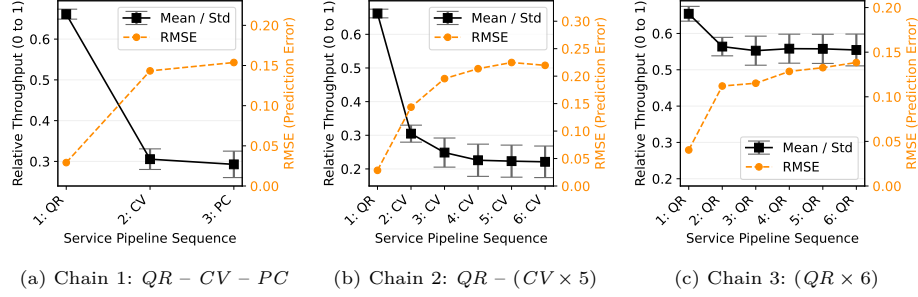


Fig. 6: Expected performance across different composed service chains; combined with the prediction error for estimating the performance of each individual service link.

Table 4: Computational overhead of training three GPs from metrics, recomposing them into arbitrary chains, and making predictions about the resulting performance.

Phase	Train models	Recompose models	Make predictions
Base GPs	2,700 ms	—	—
Chain #1	—	3 ms	80 ms
Chain #2	—	12 ms	120 ms
Chain #3	—	13 ms	120 ms

and RMSE; nevertheless, we can also see that this would be a comparably stable chain without dramatic bottlenecks.

We conclude that we were effectively able to concatenate the individual service models into larger applications and predict the performance of each individual service. This can be used to sample the expected performance across the chain, and thus, optimize the performance by descending to globally optimal configurations. However, the observed predictions were naturally noisy; to that extent, we envision an additional fine-tuning step that optimizes the GP training by allowing gradients to flow between neighbouring services.

Experiment 2.2 – Computational Overhead To support decentralized decision making across numerous distributed providers and consumers, we must guarantee low overhead for our methodology. Consequentially, we analyze the runtime overhead in Experiment 2.1 for training and recomposing the GPs, as well as for making predictions about the performance of the resulting chain. Table 4 shows the overhead captured throughout this lifecycle. We observe that the complexity is dominated by the training of the GPs, whereas the recomposition of the GPs and the actual predictions run in a fraction of that time. On the lines with the other experiments, we see reason for splitting the training into a foundational step for pretraining the base GPs, and a fine-tuning step for the composed chain. Both steps can be performed before instantiating the service chain, or while operating it to improve the accuracy over the runtime.

We summarize that the GP training dominates the complexity of our methodology; to improve this, we will look into splitting the training process, or distilling the number of samples in the GP further to speed up the GP fitting.

5 Conclusion

In this paper, we addressed the structural tension between the vast configuration space of modern computing services and the high cost of empirical benchmarking. We proposed a framework for establishing Probabilistic Performance Guarantees (PPGs) that enables service providers to offer diverse and highly customizable configurations with quantifiable confidence. By leveraging Gaussian Processes, our methodology effectively captures the inherent uncertainty of distributed environments while maintaining high sample efficiency. Furthermore, our compositional framework, built upon Decoupled Variational Inference, allows for the assembly of multi-stage service chains without compromising the architectural autonomy or data privacy of individual providers. Our empirical evaluation, conducted on a physical prototype with diverse computer vision and Lidar processing services, demonstrates that the framework accurately predicts performance and uncertainty even in unexplored areas of the solution space. Notably, we introduced the readiness score as a vital metric for decision-making, allowing providers to identify mature service offers that balance expected benefits against the risk of SLO violations. As distributed systems move toward autonomous, machine-speed service composition, such probabilistic frameworks will be essential for creating robust and trustworthy agreements between providers and consumers. In future work, we plan to extend our methodology to support complex, non-linear service directed acyclic graphs (DAGs) and investigate how the uncertainty models can be updated in real-time to incorporate new data.

Acknowledgment

First, we would like to express our gratitude to Görtler et al. [7] for helping us to improve our understanding of Gaussian processes.

References

1. de Abril, I.M., Rouska, N.M., Casamayor, V., Dustdar, S.: AICon: Agentic Intelligence for the Computing Continuum. *IEEE Internet Computing* **29**(6), 53–62 (Nov 2025). <https://doi.org/10.1109/MIC.2025.3645964>
2. Damianou, A.C., Lawrence, N.D.: Deep Gaussian Processes (Mar 2013). <https://doi.org/10.48550/arXiv.1211.0358>, arXiv:1211.0358 [stat]
3. Dzung, N.M.: Super Fast and Accurate 3D Object Detection based on 3D LiDAR Point Clouds (SFA3D) (2020), <https://github.com/maudzung/SFA3D>
4. Gan, Y., et al.: An Open-Source Benchmark Suite for Cloud and IoT Microservices (May 2019). <https://doi.org/10.48550/arXiv.1905.11055>, arXiv:1905.11055 [cs]

5. Gardner, J.R., Pleiss, G., Bindel, D., Weinberger, K.Q., Wilson, A.G.: GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration (Jun 2021). <https://doi.org/10.48550/arXiv.1809.11165>, arXiv:1809.11165 [cs]
6. Geiger, A., Lenz, P., Urtasun, R.: Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. In: Conference on Computer Vision and Pattern Recognition (CVPR) (2012)
7. Görtler, J., Kehlbeck, R., Deussen, O.: A Visual Exploration of Gaussian Processes. *Distill* **4**(4) (2019), <https://distill.pub/2019/visual-exploration-gaussian-processes>
8. Harchol-Balter, M.: Performance modeling and design of computer systems: queueing theory in action. Cambridge University Press (2013)
9. Kraft, D.: A Software Package for Sequential Quadratic Programming. *Wiss. Berichtswesen d. DFVLR* (1988)
10. Laso, S., Murturi, I., Frangoudis, P., Herrera, J.L., Murillo, J.M., Dustdar, S.: A Multidimensional Elasticity Framework for Adaptive Data Analytics Management in the Computing Continuum (Jan 2025)
11. Liu, F., Tang, G., Li, Y., Cai, Z., Zhang, X., Zhou, T.: A Survey on Edge Computing Systems and Tools. *Proceedings of the IEEE* **107**(8), 1537–1562 (Aug 2019). <https://doi.org/10.1109/JPROC.2019.2920341>
12. Nastic, S., Morichetta, A., Pusztai, T., Dustdar, S., Ding, X., Vij, D., Xiong, Y.: SLOC: Service Level Objectives for Next Generation Cloud Computing. *IEEE Internet Computing* **24**(3) (May 2020)
13. opencv: opencv at 4.9.0 (2024), <https://github.com/opencv/opencv/tree/4.9.0>
14. Pan, Z., Wang, Y., Zhang, Y., Yang, S.B., Cheng, Y., Chen, P., Guo, C., Wen, Q., Tian, X., Dou, Y., Zhou, Z., Yang, C., Zhou, A., Yang, B.: MagicScaler: Uncertainty-Aware, Predictive Autoscaling. *Proc. VLDB Endow.* **16**(12), 3808–3821 (Aug 2023). <https://doi.org/10.14778/3611540.3611566>
15. Quattrocchi, G., Incerto, E., Pincioli, R., Trubiani, C., Baresi, L.: Autoscaling Solutions for Cloud Applications Under Dynamic Workloads. *IEEE Transactions on Services Computing* **17**(3), 804–820 (May 2024)
16. Sedlak, B., Pujol, V.C., Donta, P.K., Dustdar, S.: Designing Reconfigurable Intelligent Systems with Markov Blankets. In: *Service-Oriented Computing*. pp. 42–50. Springer Nature Switzerland, Rome, Italy (2023)
17. Sedlak, B., Raith, P., Morichetta, A., Pujol, V.C., Dustdar, S.: Multi-Dimensional Autoscaling of Stream Processing Services on Edge Devices (Oct 2025). <https://doi.org/10.48550/arXiv.2510.06882>
18. Verma, S., Bala, A.: Auto-scaling techniques for IoT-based cloud applications: a review. *Cluster Computing* **24**(3) (Sep 2021). <https://doi.org/10.1007/s10586-021-03265-9>
19. Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., Ding, G.: YOLOv10: Real-Time End-to-End Object Detection (Oct 2024). <https://doi.org/10.48550/arXiv.2405.14458>, arXiv:2405.14458 [cs]
20. Wilhelm, R., Engblom, J., Ermedahl, A., et al.: The worst-case execution-time problem—overview of methods and survey of tools. *ACM Transactions on Embedded Computing Systems (TECS)* **7**(3), 1–53 (2008)
21. Zhang, C., Wang, L., He, K.: Cloud service composition optimization based on service association impact and improved NSGA-II algorithm. *Scientific Reports* **15**(1), 26001 (Jul 2025). <https://doi.org/10.1038/s41598-025-10040-y>
22. Zhao, Y., Uta, A.: Tiny Autoscalers for Tiny Workloads: Dynamic CPU Allocation for Serverless Functions. *IEEE Comp. Society* (2022)